Aim: To build and evaluate a Naive Bayes model for Network Traffic Classification using KDD dataset.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, ConfusionMatrixDisplay
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
path="/content/drive/MyDrive/ML_TAE_DATASET/kdd_network_traffic_classification A.csv"
df=pd.read_csv(path)
```

```
df.describe()
```

	duration	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	num_failed_logins	logged_
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.0000
mean	2504.761000	49908.315000	48826.014000	0.502000	1.499000	1.518000	10.122000	2.477000	0.4780
std	1443.255878	29130.464054	28066.852813	0.500246	1.111411	1.122023	6.088036	1.747147	0.4997
min	7.000000	115.000000	16.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0000
25%	1314.750000	25117.500000	25231.750000	0.000000	1.000000	0.000000	5.000000	1.000000	0.0000
50%	2443.500000	48106.000000	48062.500000	1.000000	1.000000	2.000000	10.000000	3.000000	0.0000
75%	3792.000000	75937.500000	71736.250000	1.000000	2.000000	3.000000	16.000000	4.000000	1.0000
max	4998.000000	99995.000000	99969.000000	1.000000	3.000000	3.000000	20.000000	5.000000	1.0000

8 rows × 28 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 32 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   duration            1000 non-null   int64
 1   protocol_type       1000 non-null   object
 2   service             1000 non-null   object
 3   flag                1000 non-null   object
 4   src_bytes           1000 non-null   int64
 5   dst_bytes           1000 non-null   int64
 6   land                1000 non-null   int64
 7   wrong_fragment      1000 non-null   int64
 8   urgent              1000 non-null   int64
 9   hot                 1000 non-null   int64
 10  num_failed_logins   1000 non-null   int64
 11  logged_in           1000 non-null   int64
 12  num_compromised     1000 non-null   int64
 13  root_shell          1000 non-null   int64
 14  su_attempted        1000 non-null   int64
 15  num_root            1000 non-null   int64
 16  num_file_creations  1000 non-null   int64
 17  num_shells          1000 non-null   int64
 18  num_access_files    1000 non-null   int64
 19  is_guest_login      1000 non-null   int64
 20  count               1000 non-null   int64
 21  srv_count           1000 non-null   int64
 22  serror_rate         1000 non-null   float64
 23  srv_serror_rate     1000 non-null   float64
 24  rerror_rate         1000 non-null   float64
 25  srv_rerror_rate     1000 non-null   float64
 26  same_srv_rate       1000 non-null   float64
 27  diff_srv_rate       1000 non-null   float64
 28  srv_diff_host_rate  1000 non-null   float64
 29  dst_host_count      1000 non-null   int64
 30  dst_host_srv_count  1000 non-null   int64
 31  label               1000 non-null   object
```

```
dtypes: float64(7), int64(21), object(4)
memory usage: 250.1+ KB
```

```
df.shape
```

```
(1000, 32)
```

```
df.columns
```

```
Index(['duration', 'protocol_type', 'service', 'flag', 'src_bytes',
       'dst_bytes', 'land', 'wrong_fragment', 'urgent', 'hot',
       'num_failed_logins', 'logged_in', 'num_compromised', 'root_shell',
       'su_attempted', 'num_root', 'num_file_creations', 'num_shells',
       'num_access_files', 'is_guest_login', 'count', 'srv_count',
       'serror_rate', 'srv_serror_rate', 'rerror_rate', 'srv_rerror_rate',
       'same_srv_rate', 'diff_srv_rate', 'srv_diff_host_rate',
       'dst_host_count', 'dst_host_srv_count', 'label'],
      dtype='object')
```

```
df = df.dropna()
df = pd.get_dummies(df)
X = df.iloc[:, :-1]
y = df.iloc[:, -1]
```

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.3,random_state=42)
```

```
df.isnull().sum()
df.isnull().any().any()
df[df.isnull().any(axis=1)]
```

duration	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	num_failed_logins	logged_in	num_compromised	...	flag_S0

0 rows × 49 columns

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.3,random_state=42)
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
print(confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
[[267   0]
 [  0  33]]
Accuracy: 1.0
              precision    recall  f1-score   support

       False       1.00      1.00      1.00       267
        True       1.00      1.00      1.00        33

    accuracy                           1.00       300
   macro avg       1.00      1.00      1.00       300
weighted avg       1.00      1.00      1.00       300
```

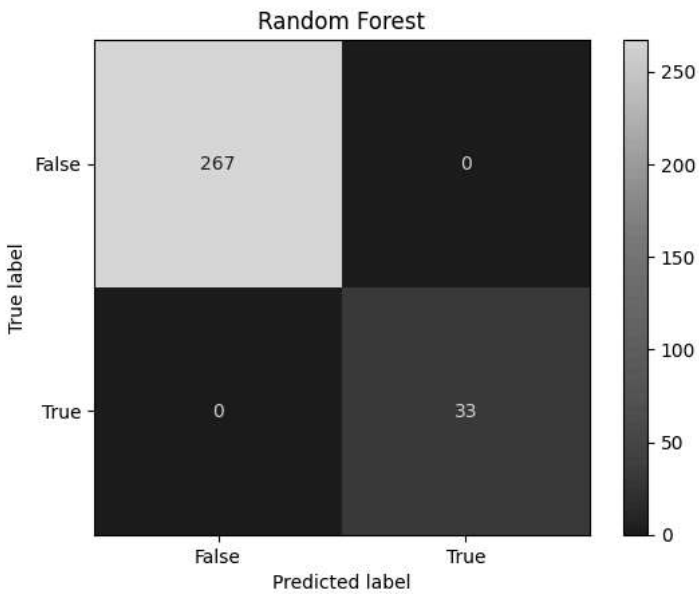
```
y_pred = model.predict(X_test)
```

```
print(confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.title("Random Forest")
plt.show()
```

```
[[267   0]
 [  0  33]]
Accuracy: 1.0
              precision    recall  f1-score   support

       False       1.00      1.00      1.00       267
        True       1.00      1.00      1.00        33

    accuracy                           1.00       300
   macro avg       1.00      1.00      1.00       300
weighted avg       1.00      1.00      1.00       300
```




```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
print("\n--- Evaluation Metrics ---")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average='weighted'))
print("Recall:", recall_score(y_test, y_pred, average='weighted'))
print("F1 Score:", f1_score(y_test, y_pred, average='weighted'))
```

```
--- Evaluation Metrics ---
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
```